Why API and types matter

Julian Geiger

LMS Seminar · 18 Mar 2026

Why should one care?

Your library's API is the **surface area** that every user touches

- A bad API leads to bugs, frustration, and misuse
- **More time is spent reading code than writing it** — API clarity pays off every day
- Code without types is implicit knowledge that lives in someone's head
- When a type is hard to write, it's often a **signal that the design needs work**

Outline

API Design

1. `urllib` vs `requests`
2. Flat is better than nested
3. Naming — short as possible, clear as necessary
4. Minimize mutable state and side effects
5. The pit of success
6. Progressive disclosure
7. Choose the right input representation

Types

1. Function signatures are your API contract
2. `stubgen` — Python's "header files"
3. Types catch bugs and guide the API
4. Types reveal code smells
5. TypedDict and dataclasses over plain dicts
6. Literals and Enums over bare strings
7. `TypeAlias` — name your complex types
8. Generics — preserving type information
9. Protocols and abstract interfaces
10. `@overload` and `@singledispatch`
11. Variance & Postel's Law / LSP

API Design

What makes an API good or bad — and how to tell the difference.

API: `urllib` vs `requests`

Example from `urllib`, part of Python `stdlib`

```
import urllib

manager = urllib.request.HTTPPasswordMgrWithDefaultRealm()
manager.add_password(
    None, "https://httpbin.org/", "usr", "pwd",
)
handler = urllib.request.HTTPBasicAuthHandler(manager)
opener = urllib.request.build_opener(handler)
response = opener.open(
    "https://httpbin.org/basic-auth/usr/pwd",
)
print(response.status)
```

Could you write this from scratch?

- **3 concepts**: manager, handler, opener
- **2 long class names**: `HTTPPasswordMgrWithDefaultRealm`, `HTTPBasicAuthHandler`
- A mystery `None` as first argument to `add_password`

A good API shouldn't require a tutorial for basic operations!

API: `urllib` vs `requests`

Well, how about that instead?

```
import requests

response = requests.get("https://httpbin.org/basic-auth/usr/pwd", auth=("usr", "pwd"))
print(response.status_code)
```

Same task, which code would you rather write? **API design matters.**

In the following slides, we'll explore what makes an API good, drawing on principles from Ben Hoyt's *Designing Pythonic library APIs*.

We'll use a fictional `fishnchips` library as a running example, alongside real examples from `aiida-core`.

Ben Hoyt — *Designing Pythonic library APIs* · benhoyt.com/writings/python-api-design/ (2023)

API: Flat is better than nested

Expose a clean API for common functionality; file structure is an implementation detail

```
# Bad — user must know internal file structure
from django.core.files.images import ImageFile  # 4 levels deep!
from django.contrib.auth.hashers import make_password
```

Re-exports in `aiida.orm` and `aiida.engine` ✅

```
# aiida/orm/__init__.py — 90+ classes re-exported
from .authinfos import *
from .comments import *
from .nodes import *
from .querybuilder import *

# User code
from aiida.orm import Int, CalcJobNode, QueryBuilder  # all flat
```

In `aiida-core`, by convention, anything not importable from the second level is **private API**.

However, in Python it's not enforced and the line gets blurry between users, workflow devs, and plugin devs.

API: Naming — short as possible, clear as necessary

The module name is already context — `import lib` then `lib.thing()`

```python
# Bad — where does order() come from?
from fishnchips import order

def eat_takeaways():
    meal = order(chips=1)
    # ordering what? from where?
```

```python
# Good — immediately clear
import fishnchips

def eat_takeaways():
    meal = fishnchips.order(chips=1)
    # obviously fish & chips
```

Don't name your function `order_fishnchips` — `fishnchips.order()` reads naturally.

Same reason `requests.get()` works — you're not meant to `from requests import get`.

Think about how you want users to interact with your library.

@mbercx: *docs-driven* development helps flesh out the API.

API: Naming — short as possible, clear as necessary

The same applies for classes — no need to repeat the class name in method names

```python
# aiida/orm/nodes/data/array/kpoints.py ❌
class KpointsData(ArrayData):
    def set_kpoints_mesh(self, mesh, ...): ...
    def get_kpoints_mesh(self, ...): ...
    def set_kpoints_mesh_from_density(self, ...): ...

# "kpoints" said three times in one line:
kpoints.set_kpoints_mesh([4, 4, 4])
```

```python
# let the class name provide context ✅
class KpointsData(ArrayData):
    def set_mesh(self, mesh, ...): ...
    def get_mesh(self, ...): ...
    def set_mesh_from_density(self, ...): ...

kpoints.set_mesh([4, 4, 4])
mesh = kpoints.get_mesh()
```

API: Minimize mutable state and side effects

Prefer returning new values over mutating inputs

```python
# Surprising — mutates the input
def normalize(data: list[float]) -> list[float]:
    max_val = max(data)
    for i in range(len(data)):
        data[i] /= max_val
    return data  # same object, mutated

raw = [1.0, 2.0, 3.0]
normed = normalize(raw)
# raw is now [0.33, 0.66, 1.0] — surprise!
```

```python
# Better — returns a new list
def normalize(data: list[float]) -> list[float]:
    max_val = max(data)
    return [x / max_val for x in data]

raw = [1.0, 2.0, 3.0]
normed = normalize(raw)
# raw is still [1.0, 2.0, 3.0] ✅
```

Tools: `tuple` over `list`, `frozenset`, `frozen=True` dataclasses

In aiida-core: Slurm scheduler mutated the caller's job list by reference — fix: `list` → `frozenset` + defensive copy (#7155 by @GeigerJ2 and @khsrali).

#7155: *Fix: Avoid mutating JobsList._polling_jobs inside slurm scheduler*

API: The pit of success

Design so the easiest path is the correct one

```
# requests verifies SSL by default — you have to opt OUT of safety
requests.get("https://api.example.com")  # SSL verified
requests.get("https://api.example.com", verify=False)  # explicit opt-out
```

Anti-pattern: making users opt IN to correctness

```
# Hypothetical bad API — unsafe by default
db.query("SELECT ...", sanitize=True)  # easy to forget sanitize=True
```

Pit of success principles:

- Safe defaults
- Unsafe options require explicit action
- Make wrong code look wrong: if misuse looks identical to correct use, the API has failed

API: Progressive disclosure

Simple things simple, complex things possible

```python
# Level 1 — one line, sensible defaults
response = requests.get("https://api.example.com/users")
```

```python
# Level 2 — add options as needed
response = requests.get(
    "https://api.example.com/users",
    params={"page": 2},
    timeout=5.0,
)
```

```python
# Level 3 — full control with a session
session = requests.Session()
session.auth = ("user", "pass")
session.headers.update({"Accept": "application/json"})
response = session.get("/users", params={"page": 2})
```

Don't front-load complexity. Let users discover features when they need them.

Tools: keyword arguments with defaults, module-level functions wrapping classes, layered APIs.

API: Progressive disclosure — in aiida-core

`core.ssh` transport — 17 options upfront ❌

```
❯ verdi computer configure core.ssh daint
User name [geiger_j]:
Port number [22]:
Look for keys [Y/n]:
SSH key file []: /home/geiger_j/.ssh/cscs/cscs-key
Connection timeout in s [60]:
Allow ssh agent [Y/n]:
SSH proxy jump []:
SSH proxy command []:
Compress file transfers [Y/n]:
GSS auth [False]:
GSS kex [False]:
GSS deleg_creds [False]:
GSS host [daint.alps.cscs.ch]:
Load system host keys [Y/n]:
Key policy (RejectPolicy, WarningPolicy, AutoAddPolicy) [RejectPolicy]:
Use login shell when executing command [Y/n]:
Connection cooldown time (s) [15.0]:
```

Most users (and developers) don't know what half of these mean — and most are just defaults.

API: Progressive disclosure — in aiida-core

Better: `core.ssh_async` (@khsrali) — leverage the user's existing SSH config ✅

```
❯ verdi computer configure core.ssh_async daint
Host as in 'ssh <HOST>' (needs to be a password-less setup in your ssh config) [daint.alps.cscs.ch]: daint
Maximum number of concurrent I/O operations [8]:
Local script to run before opening connection (path) [None]:
Type of async backend to use, `asyncssh` or `openssh` [asyncssh]:
Use login shell when executing command [Y/n]:
Connection cooldown time (s) [15.0]:
```

6 options instead of 17!

Transport reads the user's `~/.ssh/config` directly — host, key, proxy settings live where they belong:

```
# ~/.ssh/config — the user already has this
Host daint
    HostName daint.cscs.ch
    User geiger_j
    IdentityFile ~/.ssh/id_ed25519
    ProxyJump ela.cscs.ch
```

`core.ssh_async` delegates to `OpenSSH` / `AsyncSSH` and only exposes what's specific to AiiDA.

API: Choose the right input representation

Same problem, two approaches (#7069 vs #7116): control what happens on unhandled process failure

Two boolean flags (PR #7069, closed)

```
# aiida/engine/processes/workchains/restart.py
spec.input(
    'pause_for_unknown_errors',
    valid_type=orm.Bool,
    default=lambda: orm.Bool(False),
)
spec.input(
    'restart_once_for_unknown_errors',
    valid_type=orm.Bool,
    default=lambda: orm.Bool(True),
)
```

One string input (PR #7116, by @mbercx, merged)

```
# aiida/engine/processes/workchains/restart.py
spec.input(
    'on_unhandled_failure',
    valid_type=orm.Str,
    required=False,
    validator=validate_on_unhandled_failure,
)
# options: 'abort', 'pause',
#   'restart_once', 'restart_and_pause'
```

Two booleans → **2 extra levels of nesting** to resolve 4 combinations. One string → flat `if/elif` dispatch, each case self-contained.

#7069: *Pause BaseRestartWorkChain on unhandled errors* · #7116: *Add on_unhandled_failure input*

Types

A good API tells users *what* to pass and *what* to expect back.

Types make that contract explicit and machine-checkable — if you actually run the type checker.

Types: Function signatures are your API contract

The parallel to C header files

In C, the **header file** declares the contract — types in, types out:

```c
// shop.h — this IS the API
Receipt order(int chips, int fish, float timeout);
```

In Python, **type-annotated function signatures** serve the same purpose:

```python
# shop.py — the signature IS the documentation
def order(chips: int, fish: int, timeout: float = 10.0) -> Receipt: ...
```

- Users read signatures **before** reading docstrings
- IDEs show signatures on hover — they're the first thing people see
- A well-typed signature answers: *what do I pass in, what do I get back?*
- **If your signature needs a paragraph to explain, redesign the API**

Types: `stubgen` — Python's "header files"

Generate `.pyi` stub files that show only the API surface

- `stubgen` (from mypy) or `pyright --createstub` auto-generates them

```
# Generate stubs for your package
stubgen -p mypackage -o stubs/
```

- `.pyi` files are like C header files — types and signatures, no implementation
- Great for reviewing your public API at a glance
- Friendlier to LLM context than the full implementation ☺

Types: `stubgen` — aiida-core example

`Waiting` — the state that manages a `CalcJob` while it runs on the remote computer (~280 lines, excerpt):

```
# stubs/aiida/engine/processes/calcjobs/tasks.pyi
class Waiting(plumpy.process_states.Waiting):
    def __init__(
        self, process: CalcJob, done_callback: Callable[..., Any] | None,
        msg: str | None = None, data: Any | None = None,
    ) -> None: ...
    @property
    def monitors(self) -> CalcJobMonitors | None: ...
    @property
    def process(self) -> CalcJob: ...
    def upload(self) -> Waiting: ...
    def submit(self) -> Waiting: ...
    def update(self) -> Waiting: ...
    def stash(self, monitor_result: CalcJobMonitorResult | None = None) -> Waiting: ...
    def retrieve(self, monitor_result: CalcJobMonitorResult | None = None) -> Waiting: ...
    def parse(
        self, retrieved_temporary_folder: str, exit_code: ExitCode | None = None,
    ) -> plumpy.process_states.Running: ...
    def interrupt(self, reason: Any) -> plumpy.futures.Future | None: ...
```

The stub reveals the lifecycle at a glance: upload → submit → update → retrieve → parse.

Types: Types catch bugs and guide the API

Adding types to existing code reveals hidden issues

```python
# Before — "works fine" (or does it?)
def get_user_name(user_id):
    user = db.find(user_id)
    if user:
        return user.name
    return None  # caller might not handle this
```

```python
# After — the issue now becomes obvious
def get_user_name(user_id: int) -> str:
    # mypy error: [return-value]
    user = db.find(user_id)
    if user:
        return user.name
    return None  # None is not str
```

The type checker **forces you to decide**: return `str | None`? Or raise on missing users? Either way, the caller now knows what to expect.

In aiida-core: adding types to the scheduler module revealed wrong return types, `str` assigned where `int` was expected, and silent `None` returns (#7156 by @danielhollas).

Annotate immediately, fix flaws, avoid releasing unpolished API to avoid being stuck because of backwards compatibility.

#7156: *Add typing to scheduler plugins*

Types: Types reveal code smells

When the return type is too broad, your code needs refactoring

```python
def process_payment(
    order: Order
) -> str | int | dict | None:
    if order.method == "card":
        return {"tx_id": "abc", "status": "ok"}
    elif order.method == "cash":
        return 42  # receipt number
    elif order.method == "credit":
        return "pending"
    return None
```

```python
@dataclass
class PaymentResult:
    tx_id: str
    status: str
    receipt_number: int


# one clear return type,
# one clear responsibility
def process_payment(
    order: Order,
) -> PaymentResult: ...
```

If you struggle to write the return type, **the API is missing an abstraction**.

Types: Types reveal code smells — aiida-core

Breaking the dict contract: `None` instead of `KeyError` (surfaced in #7136 by @danielhollas)

```python
# aiida/common/extendeddicts.py — the root cause
class DefaultFieldsAttributeDict(AttributeDict):
    def __getitem__(self, key: str) -> Any | None:
        try:
            return super().__getitem__(key)
        except KeyError:
            if key in self._default_fields:
                return None  # breaks the dict contract
                             # — KeyError expected!
            raise
```

```python
# aiida/schedulers/datastructures.py — victim
class JobInfo(DefaultFieldsAttributeDict):
    if TYPE_CHECKING:
        num_machines: int        # int (not Optional)...
        allocated_machines: list[MachineInfo]  # same
```

```python
# aiida/schedulers/plugins/slurm.py
# field may be None at runtime, but type checker
# sees int — contradiction:
if (
    this_job.num_machines is not None
    and this_job.allocated_machines is not None
):  # type: ignore[redundant-expr]
    ...
```

#7136: *Strict typing for aiida.schedulers module*

Types: TypedDict and dataclasses over plain dicts

Structure your data, don't just bag it

```
# The "stringly typed" approach — no safety, no autocomplete
user = {"name": "Alice", "age": 30, "emial": "alice@example.com"}
#                                   ^^^^^ typo goes unnoticed
```

```
# TypedDict — lightweight, catches key typos
from typing import TypedDict


class User(TypedDict):
    name: str
    age: int
    email: str


user: User = {"name": "Alice", "age": 30, "emial": "..."}
#                                          ^^^^^^ error!
```

```
# dataclass — type safety + methods + immutability
# (also: NamedTuple for lightweight immutable records)
from dataclasses import dataclass


@dataclass(frozen=True)
class User:
    name: str
    age: int
    email: str
```

Types: Literals and Enums over bare strings

Constrain your inputs

```python
# Fragile — any typo silently passes
def set_log_level(level: str) -> None: ...

set_log_level("wraning")  # silent misbehavior
```

```python
# Better — Literal constrains the values
from typing import Literal

LogLevel = Literal[
    "debug", "info", "warning", "error",
]

def set_log_level(level: LogLevel) -> None: ...

set_log_level("wraning")  # type error!
```

```python
# Best — Enum with exhaustiveness
from enum import Enum
from typing import assert_never

class LogLevel(Enum):
    DEBUG = "debug"
    # INFO = "info"
    # Adding this will break if not explicitly handled

def set_log_level(level: LogLevel) -> None:
    match level:
        case LogLevel.DEBUG: ...
        case _:
            assert_never(level)  # error!
```

Adding a new member → every `match` that forgot it becomes a type error.

Types: Literals and Enums — in aiida-core

```python
# aiida/common/links.py
class LinkType(Enum):
    CREATE = 'create'
    RETURN = 'return'
    INPUT_CALC = 'input_calc'
    INPUT_WORK = 'input_work'
    CALL_CALC = 'call_calc'
    CALL_WORK = 'call_work'

# aiida/common/datastructures.py
class CalcJobState(Enum):
    UPLOADING = 'uploading'
    SUBMITTING = 'submitting'
    WITHSCHEDULER = 'withscheduler'
    RETRIEVING = 'retrieving'
    PARSING = 'parsing'
```

```python
# aiida/orm/entities.py
class EntityTypes(Enum):
    AUTHINFO = 'authinfo'
    COMMENT = 'comment'
    COMPUTER = 'computer'
    GROUP = 'group'
    LOG = 'log'
    NODE = 'node'
    USER = 'user'
    LINK = 'link'

# aiida/common/datastructures.py
class StashMode(Enum):
    COPY = 'copy'
    COMPRESS_TAR = 'tar'
    COMPRESS_TARBZ2 = 'tar.bz2'
    COMPRESS_TARGZ = 'tar.gz'
    COMPRESS_TARXZ = 'tar.xz'
```

Types: `TypeAlias` — name your complex types

Tame your type annotations

```
# Without aliases — good luck reading this
def handle(
    callback: Callable[[Request, dict[str, list[str]]], Awaitable[Response | None]],
    middlewares: list[Callable[[Request], Awaitable[Request]]],
) -> None: ...
```

```
# With aliases — intent is clear
from typing import TypeAlias

Headers: TypeAlias = dict[str, list[str]]
Handler: TypeAlias = Callable[[Request, Headers], Awaitable[Response | None]]
Middleware: TypeAlias = Callable[[Request], Awaitable[Request]]


def handle(callback: Handler, middlewares: list[Middleware]) -> None: ...
```

- Aliases are **documentation** — they name a concept, not just a shape
- Reusable across your codebase — change the definition in one place

Types: `TypeAlias` — in aiida-core

graph traversal module uses aliases for complex types ✅

```
# aiida/tools/graph/age_entities.py
from typing_extensions import TypeAlias

_NodeOrGroupCls: TypeAlias = 'type[orm.Node] | type[orm.Group]'
_ContainerTypes: TypeAlias = 'list[Any] | tuple[Any, ...] | set[Any]'
_EdgeType: TypeAlias = 'type[LinkQuadruple] | type[GroupNodeEdge]'
_BasketKeys: TypeAlias = Literal['nodes', 'groups', 'nodes_nodes', 'groups_nodes']
```

`CalcJobNode` repeats the same complex type 5 times ❌

```
# aiida/orm/nodes/process/calculation/calcjob.py
def _validate_retrieval_directive(
    directives: Sequence[Union[str, Tuple[str, str, str]]]) -> None: ...

def set_retrieve_list(
    self, retrieve_list: Sequence[Union[str, Tuple[str, str, str]]]) -> None: ...

def get_retrieve_list(
    self) -> Optional[Sequence[Union[str, Tuple[str, str, str]]]]: ...

# Should be:  RetrievalDirective: TypeAlias = Sequence[str | tuple[str, str, str]]
```

Types: Generics — preserving type information

Don't lose type precision through transformations

```python
# Without generics — loses the element type
def first(items: list[Any]) -> Any:
    return items[0]


val = first([1, 2, 3])  # val: Any
val.upper()  # no error — crashes at runtime!
```

```python
# With generics — return type tracks input
from typing import TypeVar

T = TypeVar("T")


def first(items: list[T]) -> T:
    return items[0]


val = first([1, 2, 3])  # val: int
val.upper()  # error: int has no .upper()
```

Generics keep the **type checker informed** across function boundaries.

Types: Protocols and abstract interfaces

Depend on behavior, not implementation

```python
from typing import Protocol


class Readable(Protocol):
    def read(self, n: int = -1) -> bytes: ...


def process_data(source: Readable) -> None:
    data = source.read()
    ...
```

Any object with a `.read()` method satisfies `Readable` — **no inheritance required**.

- **ABCs** = nominal subtyping (must inherit). **Protocols** = structural subtyping (just match the shape).
- Makes code **testable** — pass a fake that matches the protocol

Types: `@overload` and `@singledispatch`

`@overload` — narrow return types statically

```
# aiida/orm/querybuilder.py
@overload
def first(
    self, flat: Literal[False] = ...,
) -> list[Any] | None: ...
@overload
def first(
    self, flat: Literal[True],
) -> Any | None: ...

def first(
    self, flat: bool = False,
) -> list[Any] | Any | None: ...
```

No `@overload`, always `list[Any] | Any | None`.

`@singledispatch` — runtime dispatch by type

```
# aiida/orm/convert.py
@singledispatch
def get_orm_entity(backend_entity):
    raise TypeError(...)

@get_orm_entity.register(BackendComputer)
def _(backend_entity):
    return Computer.from_backend_entity(backend_entity)
```

Replaces `isinstance` chains — new types register handlers without modifying the function.

Types: Variance, Postel's Law and LSP

Variance — why `list[Dog]` is not `list[Animal]`

- **Mutable** (`list`) → **invariant**: allowing `list[Dog]` as `list[Animal]` lets you `append(Cat())` — your dog list now has a cat
- **Read-only** (`Sequence`) → **covariant**: no mutation possible, so `Sequence[Dog]` *is* `Sequence[Animal]`

Postel's Law — "Be liberal in what you accept, be conservative in what you send" (RFC 761)

- Accept broad input types (`Sequence` , `Mapping`), return narrow types (`list` , `dict`)

Liskov Substitution Principle (LSP) — a subclass must accept everything its parent accepts. Narrowing parameter types breaks callers at runtime.

- **In aiida-core:** `CalcJob.define(spec: CalcJobProcessSpec)` narrows from `Process.define(spec: ProcessSpec)` — suppressed with `type: ignore[override]`

Types: `type: ignore` in aiida-core `v2.8.0`

(Still) 367 `type: ignore` across the codebase (343 in `src/`, 24 in tests)

Category	Count	What it reveals
`attr-defined`	94	Heavy dynamic attribute access (ORM, mixins)
`arg-type`	43	API polymorphism that types can't express
`union-attr`	31	Missing type narrowing
`return-value`	27	Return type doesn't match annotation
`assignment`	27	Variables change type mid-function
`override`	18	Inheritance hierarchy violates LSP

Each `type: ignore` is a **decision**: fix the design, or document the debt.

Putting it all together — API design

Principle	Benefit
Flat imports	File structure is an implementation detail
Naming	Short as possible, clear as necessary
Immutability	Fewer surprises, thread safety
Pit of success	Safe defaults, explicit opt-out
Progressive disclosure	Simple things simple, complex things possible
Input representation	Right abstraction reduces nesting

Putting it all together — Type system

Tool	Benefit
Typed signatures	Machine-checkable contracts
dataclass / TypedDict	Structured data, typo detection
Enums / Literals	Constrained inputs, exhaustiveness
Protocols	Flexible abstractions, testability
Generics	Preserve type info across boundaries
@overload / @singledispatch	Precise dispatch by type
Variance / LSP	Subtyping done right

Key takeaway

A good API is easy to type. Well-typed code is well-designed code.

They enforce each other — types surface design flaws, and clean design makes types simple.

Conclusion

Rewrite it in Rust

🦀

Alternative conclusion

Time for `aiida-core v3`?!

References

- Ben Hoyt — *Designing Pythonic library APIs* (2023)
- Reviewing typing PRs by Daniel Hollas
- `Claude Code`

Relevant PEPs

- PEP 484 — Type Hints (`@overload` , generics, variance)
- PEP 544 — Protocols (structural subtyping)
- PEP 567 — Context Variables (`ContextVar`)
- PEP 586 — Literal Types
- PEP 589 — TypedDict
- PEP 612 — ParamSpec (decorator signatures)
- PEP 613 — TypeAlias

Thank you!

Powered by Slidev